

Database Management System

Relational Model

Antu Chowdhury
Lecturer
Dept. of CSE, EWU

Relational Model Concepts

The **relational model** stores a database as a **collection of tables**.

- Each **table** is called a **relation**.
- Each **row** in a table represents **one real-world object or event**.
- Each **column** represents a **type of information** about that object.

Student Table

Student_ID	Name	Department
101	Rahim	CSE
102	Karim	EEE

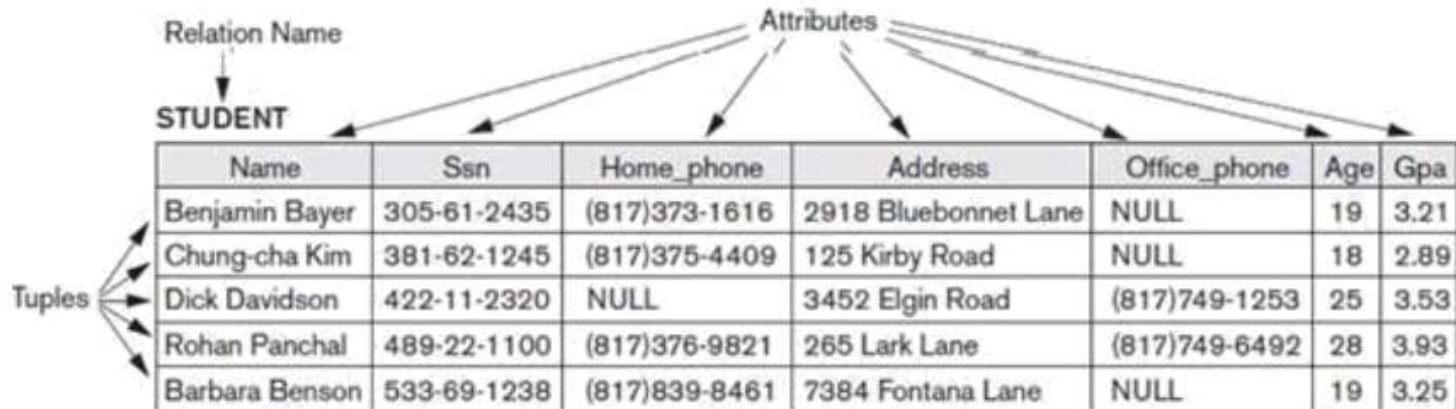
- The **table name** Student tells us what the data is about.
- Each **row** represents **one student** (a real-world entity).
- Each **column name** explains what kind of value is stored (ID, name, department).

Relational Model Concepts

In the formal relational model terminology,

- ❑ a row is called a tuple,
- ❑ a column header is called an attribute,
- ❑ the table is called a relation.

The data type describing the types of values that can appear in each column is represented by a **domain** of possible values.



Domains, Attributes, Tuples, and Relations

- ❑ The set of allowed values for each attribute is called the **domain** of the attribute.
- ❑ Some examples of domains follow:
 - ❑ BD_phone_numbers: The set of 11-digit phone numbers valid in BD.
 - ❑ Employee_ages: Possible ages of employees in a company; each must be an integer value between 15 and 80.
- ❑ Attribute values are (normally) required to be **atomic**; that is, single and indivisible. E.g. the value of an attribute can be an account number but cannot be a set of account numbers. Full name attribute can be split into first name , last name to ensure atomicity.
- ❑ The special value **null** is a member of every domain. Indicated that the value is “unknown”

Relations are Unordered

- ❑ Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- ❑ Example: instructor relation with unordered tuples

The picture can't be displayed.

Relation Schema

- ❑ A relation schema R ,
 - ❑ $R = (A_1, A_2, \dots, A_n)$ is a *relation schema* where $A_1, A_2, \dots, A_n$ are *attributes*
 - ❑ Example:
 - ❑ *instructor* = (*ID*, *name*, *dept_name*, *salary*)
 - ❑ *Customer_schema* = (*customer_name*, *customer_street*, *customer_city*)
- ❑ A **relation instance** is the **actual data stored in the table at a particular time**.
Denoted as: $r(R)$
- ❑ This means a relation r that follows the schema R .
- ❑ Example:
 - ❑ *customer* (*Customer_schema*)
- ❑ A **tuple** is a **single row in a relation instance**.

Database

- ❑ A database consists of multiple relations
- ❑ Information about an enterprise is broken up into parts, with each relation storing one part of the information
- ❑
 - account: stores information about accounts
 - depositor : stores information about which customer owns which account
 - customer : stores information about customers
- ❑ Storing all information as a single relation such as
bank(account_number, balance, customer_name, ..) results in repetition of information e.g., if one customer owns several accounts (What gets repeated?)
- ❑ the need for null values e.g., to represent a customer without an account
- ❑ **Normalization theory** explains **how to design tables properly** to avoid these problems.

Key Constraints and Constraints on NULL Values

A key is a set of one or more attributes that can uniquely identify each row in a table. A key not only identifies the rows of a table but also relates two or more tables.

Different Types of Keys:

- 1) Super Key
- 2) Candidate Key
- 3) Primary Key
- 4) Foreign Key
- 5) Secondary Key/Alternate Key
- 6) Unique Key
- 7) Composite Key

Key Constraints – Super Key

- ❑ K is a **superkey** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.

<Student>

Student_ID	Student_Enroll	Student_Name	Student_Email
S02	4545	Dave	ddd@gmail.com
S34	4541	Jack	jjj@gmail.com
S22	4555	Mark	mmm@gmail.com

Key Constraints – Super Key

- ❑ K is a **superkey** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.

Super Keys are:

{Student_ID}

{Student_Enroll}

{Student_Email}

{Student_ID, Student_Enroll}

{Student_ID, Student_Name}

{Student_ID, Student_Email}

{Student_Name, Student_Enroll}

{Student_ID, Student_Enroll, Student_Name}

{Student_ID, Student_Enroll, Student_Email}

{Student_ID, Student_Enroll, Student_Name, Student_Email}

Key Constraints – Candidate Key

- ❑ Superkey K is a **candidate key** if K is minimal

Candidate Keys are:

{Student_ID}

{Student_Enroll}

{Student_Email}

Key Constraints – Primary Key

- ☐ One of the candidate keys is selected to be the **primary key**.
 - ☐ Which one?
 - ☐ Should choose an attribute whose value never, or very rarely, changes.
 - ☐ E.g. email address is unique, but may change

- ☐ The main difference between the primary key and the candidate key is that the primary key does not contain NULL values.

- ✓ Primary Key must be UNIQUE and NOT NULL.

Key Constraints – Foreign Key

- ❑ A foreign key is a set of attributes in a table that refers to the primary key of another table.
- ❑ Only values occurring in the primary key attribute of the referenced relation may occur in the foreign key attribute of the referencing relation.

Primary Key



SROLL_NO	SNAME	DOB
12	Keerthi	10-03-1999
13	Siddhartha	18-06-2001
14	Krishna	21-09-2000
15	Suma	30-01-1998

Foreign Key



SROLL_NO	PROJECT_ID	MARKS	STATUS
12	P1	100	Pass
13	P2	90	Pass
14	P3	80	Pass

Key Constraints – Secondary Key

- ❑ A primary key is the field in a database that is the primary key used to uniquely identify a record in a database. A secondary key is an additional key, or alternate key, which can be use in addition to the primary key to locate specific data.
- ❑ Secondary Key is the key that has not been selected to be the primary key. However, it is considered a candidate key for the primary key.
- ❑ Note: Secondary Key is not a Foreign Key.

Student_ID	Student_Enroll	Student_Name	Student_Age	Student_Email
096	9122717	Manish	25	aaa@gmail.com
055	9122655	Manan	23	abc@gmail.com
067	9122699	Shreyas	28	pqr@gmail.com

- ❑ Student_ID, Student_Enroll and Student_Email are the candidate keys. Select any one of the candidate key as the primary key. Rest of the two keys would be Secondary Key.

Key Constraints – Unique Key

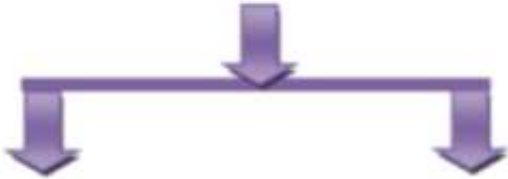
- ❑ A Unique Key is used to prevent duplicate values in a column.
- ❑ A primary key cannot accept NULL values; this makes Primary Key different from unique Key, since Unique Key allows one value as NULL value.
- ❑ A table can only have a single Primary Key, whereas a Unique Key can be more than one if you need it in the table.
- ❑ Unique Key ensures that data is not duplicated in two rows in the database. A row in the database can have null in case of Unique Key.
- ❑ You cannot modify a Primary Key, but a Unique Key can be modified.

Key Constraints – Composite Key

- ❑ A primary key having two or more attributes is called composite key. It is a combination of two or more columns

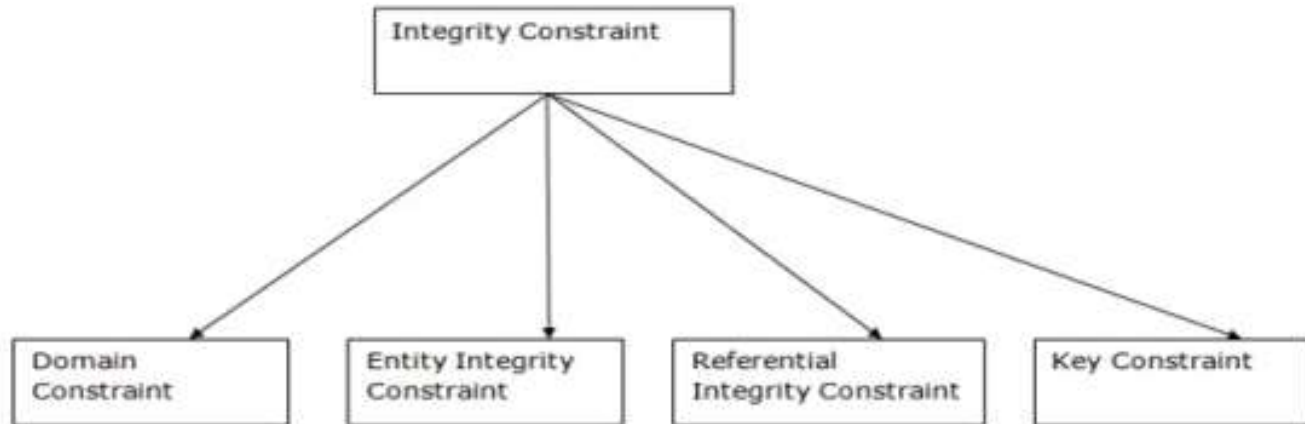
Example 1: Here our composite key is OrderID and ProductID – {OrderID, ProductID}

Composite Key



OrderID	ProductID	Quantity
22324	99	4
11332	99	9
23467	145	7
22324	129	3

Integrity, Referential Integrity, and Foreign Keys



Integrity, Domain Constraints

Domain Constraints:

- ✓ Domain constraints can be defined as the definition of a valid set of values for an attribute.
- ✓ The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

Integrity, Entity Integrity Constraints

Entity integrity constraints

- ✓ The entity integrity constraint states that primary key value can't be null.
- ✓ This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- ✓ A table can contain a null value other than the primary key field

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

Integrity, Referential Integrity Constraints

Referential Integrity Constraints

- ✓ A referential integrity constraint is specified between two tables.
- ✓ In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2

